

Project 3 – IFC and Noninterference

Due | 08.07.2026

TU Wien

1 General Remarks

This project focuses on the implementation of the Horn-clause-based abstraction of the `nia` rules presented during the course. It consists in implementing a python program to generate Z3 formulas that can be checked by the solver to detect noninterference.

2 Setting up Your System

This homework consists in developing a Python program that uses the Z3 theorem prover as a library.

- The project template requires Python version ≥ 3.12 .
- You can install Z3 and its python bindings using `pip` with the following command.

```
1 pip install z3-solver
```

- The project template includes a `requirements.txt` file defining all required dependencies that can be installed in a virtual environment with the following commands.

```
1 python3 -m venv .venv
2 source .venv/bin/activate
3 pip install -r requirements.txt
```

3 Submission Instructions

Please submit your solution by *08.07.2026* on TUWEL. This project will account for 18 out of the 60 project points.

Your submission must be a single archive file `name-matriculation.zip` containing

1. The `submission.pdf` file containing all your answers and explanations, a $\text{\LaTeX}$ template can be found on TUWEL.
2. The python files containing your solutions, as described in the submission box of each task:

Required files

<code>nia.py</code>

<code>nia-abstract.py</code>

4 Noninterference Analysis

Your main task for this project is to implement a version of the value-sensitive noninterference analysis presented in the lecture. As the analysis presented in class is meant to handle arbitrary programming language semantics, it contains a level of complexity unnecessary for our **WHILE** language.

For this reason, in this project we will use the simplified set of rules for **nia** presented in Figure 1. Notably, we do not need the **RAISE-CONTEXT-NO-STEP** rule. Additionally, since control-flow and joining instructions do not modify the memory, we removed the logic to update memories and label maps from the **RAISE-CONTEXT-STEP** and **JOIN** rules. Finally, as in **WHILE** every structured control flow instruction can be uniquely associated with an instruction that executes the joining transition, we simplified the context object to only hold the program position of the instruction that will execute the join.

$$\begin{array}{c}
 \text{INIT} \\
 \hline
 \{(\ominus_{ctx}, pp, \Lambda_0, m_1), (\ominus_{ctx}, pp, \Lambda_0, m_2)\} \subseteq \text{nia}(\Lambda_0, pp, m_1, m_2) \\
 \\
 \text{PROPAGATE-TAINT} \\
 \hline
 \begin{array}{l}
 (\zeta, pp', \Lambda, m') \in \text{nia}(\Lambda_0, pp, m_1, m_2) \\
 \zeta = (\zeta_L, pp_j) \quad (pp'', I_{pp}, U) \in \text{astep}(pp', m') \quad \Lambda' = \lambda p. \begin{cases} \bigsqcup_{i \in I_U} \Lambda(i) \sqcup \zeta_L & \text{if } (p, _, I_U) \in U \\ \Lambda(p) & \text{otherwise} \end{cases} \\
 pp_j \neq pp' \vee \zeta_L = \ominus \quad \neg \text{ccf}(pp') \vee \zeta_L = \textcircled{?} \vee \bigsqcup_{i \in I_{pp}} \Lambda(i) = \ominus
 \end{array} \\
 \hline
 \{(\zeta, pp'', \Lambda', \text{apply}(m', U))\} \subseteq \text{nia}(\Lambda_0, pp, m_1, m_2) \\
 \\
 \text{RAISE-CONTEXT-STEP} \\
 \hline
 \begin{array}{l}
 (\ominus_{ctx}, pp', \Lambda, m') \in \text{nia}(\Lambda_0, pp, m_1, m_2) \quad (pp'', I_{pp}, _) \in \text{astep}(pp', m') \\
 pp_j = \text{joinat}(pp') \quad \zeta' = (\textcircled{?}, pp_j) \quad \textcircled{?} = \bigsqcup_{i \in I_{pp}} \Lambda(i) \quad \text{ccf}(pp')
 \end{array} \\
 \hline
 \{(\zeta', pp'', \Lambda, m')\} \subseteq \text{nia}(\Lambda_0, pp, m_1, m_2) \\
 \\
 \text{JOIN} \\
 \hline
 \begin{array}{l}
 ((\textcircled{?}, pp'), pp', \Lambda_1, m'_1) \in \text{nia}(\Lambda_0, pp, m_1, m_2) \quad (pp'', _, _) \in \text{astep}(pp'_1, m'_1) \\
 ((\textcircled{?}, pp'), pp', \Lambda_2, m'_2) \in \text{nia}(\Lambda_0, pp, m_1, m_2) \quad (pp'', _, _) \in \text{astep}(pp'_2, m'_2) \\
 \Lambda' = \lambda p. \begin{cases} \textcircled{?} & \text{if } (\Lambda_1(p) \sqcup \Lambda_2(p) = \textcircled{?}) \wedge m'_1(p) \neq m'_2(p) \\ \ominus & \text{otherwise} \end{cases}
 \end{array} \\
 \hline
 \{(\ominus_{ctx}, pp'', \Lambda', m'_1), (\ominus_{ctx}, pp'', \Lambda', m'_2)\} \subseteq \text{nia}(\Lambda_0, pp, m_1, m_2)
 \end{array}$$

Figure 1: Simplified rule set for **nia**

4.1 Translating nia to Horn-Clause-Based Abstractions

A program c is noninterferent, if the following formula holds:

$$\begin{aligned} & \forall m_1 m_2 m'_1 m'_2. \\ & m_1 =_{\ell}^{\Gamma} m_2 \wedge \langle c, m_1 \rangle \xrightarrow{*} \langle \epsilon, m'_1 \rangle \wedge \langle c, m_2 \rangle \xrightarrow{*} \langle \epsilon, m'_2 \rangle \\ & \implies m'_1 =_{\ell}^{\Gamma} m'_2 \end{aligned}$$

The way **nia** was presented in the lecture relied on the following assumptions:

- **nia** will be called with two memories (or abstracted memories) that are low-equal;
- the user has to take care to cover *the whole possible input space* to prove the program is noninterferent *for all inputs*.

To formalize these concepts, let's revisit **nia**'s soundness statement:

If we run a program c with two memories m_1 and m_2 , that are low-equal¹ w.r.t. a security policy Γ and an attacker ℓ , and we end up with two memories m'_1 and m'_2 that disagree on any value p , **nia** will be able to derive (from c , m_1 , m_2 and the label map Λ_{ℓ}^{Γ} corresponding to ℓ and Γ) a label map for either m'_1 or m'_2 , which labels p as $\textcircled{?}$.

$$\begin{aligned} & \forall m_1 m_2 m'_1 m'_2 c. \\ & m_1 =_{\ell}^{\Gamma} m_2 \wedge \langle c, m_1 \rangle \xrightarrow{*} \langle \epsilon, m'_1 \rangle \wedge \langle c, m_2 \rangle \xrightarrow{*} \langle \epsilon, m'_2 \rangle \wedge m'_1(p) \neq m'_2(p) \implies \\ & \exists \zeta m' \Lambda. (\zeta, \epsilon, \Lambda, m') \in \text{nia}(\Lambda_{\ell}^{\Gamma}, c, m_1, m_2) \wedge \Lambda(p) = \textcircled{?} \wedge m' \in \{m'_1, m'_2\} \end{aligned}$$

In order for $m'_1 \neq_{\ell}^{\Gamma} m'_2$ to hold, there has to be a memory position p with $\Gamma(p) \sqsubseteq \ell$ with $m'_1(p) \neq m'_2(p)$.

This means we can reformulate the soundness statement as “if from two low-equal memories m_1 and m_2 **nia** cannot derive a taint for a memory position that is visible to the attacker, then the two resulting memories are low-equal as well.”

$$\begin{aligned} & \forall m_1 m_2 m'_1 m'_2. \\ & m_1 =_{\ell}^{\Gamma} m_2 \wedge \langle c, m_1 \rangle \xrightarrow{*} \langle \epsilon, m'_1 \rangle \wedge \langle c, m_2 \rangle \xrightarrow{*} \langle \epsilon, m'_2 \rangle \wedge \\ & \neg \left(\exists \zeta m' \Lambda p. (\zeta, \epsilon, \Lambda, m') \in \text{nia}(\Lambda_{\ell}^{\Gamma}, c, m_1, m_2) \wedge p \sqsubseteq \Gamma(\ell) \wedge \Lambda(p) = \textcircled{?} \wedge m' \in \{m'_1, m'_2\} \right) \\ & \implies m'_1 =_{\ell}^{\Gamma} m'_2 \end{aligned}$$

For a set² of memory configurations $\mathbf{m}$, we write $\text{nia}(\Lambda_{\ell}^{\Gamma}, c, \mathbf{m})$ to mean $\bigcup_{m_1 \in \mathbf{m}, m_2 \in \mathbf{m}} \text{nia}(\Lambda_{\ell}^{\Gamma}, c, m_1, m_2)$.

If all memories in $\mathbf{m}$ are pairwise low-equal, i.e., $\forall m_1 \in \mathbf{m}, m_2 \in \mathbf{m}. m_1 =_{\ell}^{\Gamma} m_2$, and $\neg \exists \Lambda p. (_, \epsilon, \Lambda, _) \in \text{nia}(\Lambda_{\ell}^{\Gamma}, c, \mathbf{m}) \wedge \Lambda(p) = \textcircled{?} \wedge \Gamma(p) \sqsubseteq \ell$, we know that for the inputs in $\mathbf{m}$, c does not interfere.

If we take a partition of the input space of memories $\mathbf{M} \subseteq \mathcal{P}(\text{Mem})$, such that $\bigcup_{\mathbf{m} \in \mathbf{M}} \mathbf{m} = \text{Mem}$ and $\forall \mathbf{m} \in \mathbf{M}, m_1 \in \mathbf{m}, m_2 \in \mathbf{m}. m_1 =_{\ell}^{\Gamma} m_2$, then $\forall \mathbf{m} \in \mathbf{M}. \neg \exists \Lambda p. (_, \epsilon, \Lambda, _) \in \text{nia}(\Lambda_{\ell}^{\Gamma}, c, \mathbf{m}) \wedge \Lambda(p) = \textcircled{?} \wedge \Gamma(p) \sqsubseteq \ell$ implies that c is noninterferent.

¹We chose the slightly more verbose notation $=_{\ell}^{\Gamma}$ here (instead of $=_{\ell}$) to make the dependency to a specific security policy Γ explicit.

²In the spirit of abstract interpretation, we assume these sets to be efficiently representable.

To encode this into a Horn-clause-based abstraction, to be solved by a CHC-Solver, we encode the output of *nia* as derivable predicates. We note, that the only rule where the initial memories m_1 and m_2 are important in *nia*'s rules is JOIN. In particular, JOIN only relates runs that started out with the same memories m_1 and m_2 , which are assumed to be low-equal. In other words, when applying JOIN, we have to decide if the two facts we are currently looking at, were derived using two memories m_1, m_2 that were taken from an **m** as described above (i.e. one where all elements are low-equal). To identify the sets of initial memories, we can add the initial memories to the facts derived by the Horn-clause-based abstraction and only apply the JOIN-rule when the initial memories are low-equal.

4.1.1 From WHILE to Horn clauses

More concretely, we first map each **WHILE** statement to a *nia* rule as depicted in Figure 2.

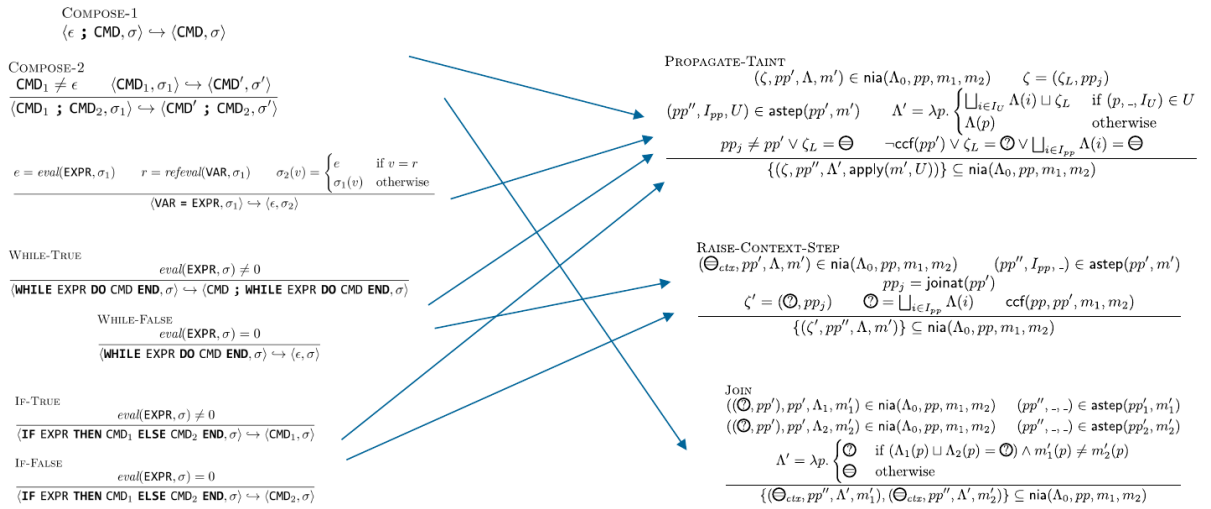


Figure 2: Mapping **WHILE** statements to *nia* rules.

Then, similarly to the approach we used for the reachability analysis (see `alpha.py`), we define two reachability predicates for each program statement, S_{pc} and E_{pc} , respectively modeling the program configuration before and after executing the statement at program counter pc . These predicates are used to encode *nia* as a Horn-clause-based reachability analysis. In particular, we include the input and output parameters of *nia* as the parameters of S and E as shown in Figure 3. Note that since our **WHILE** language doesn't support procedure calls, we do not need to add the initial label map to the arguments of the predicates.

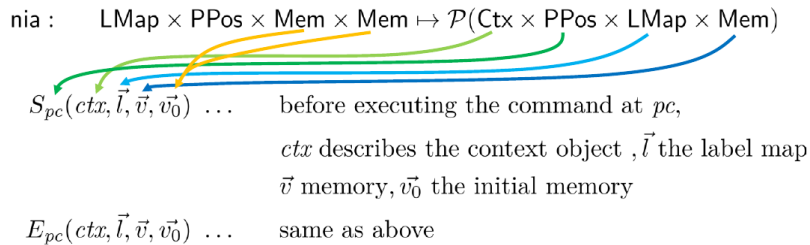


Figure 3: Mapping *nia* to the S_{pc} and E_{pc} predicates.

For each program statement, we construct set of Horn clauses that connect S_{pc} to E_{pc} depending on the constraints specified in the **WHILE** semantics and the corresponding **nia** rules. Each call to **nia** in the hypothesis or conclusion of a rule map to a S/E_{pc} predicate depending on the semantics of the specific statement we are encoding. For instance, to model variable assignment, we apply the PROPAGATE-TAINT rule starting from the state before the assignment S_{pc} (the **nia** in the hypotheses) to the state after the assignment E_{pc} (the **nia** of the conclusion of the PROPAGATE-TAINT rule). For if statements, instead, we apply the RAISE-CONTEXT-STEP or the PROPAGATE-TAINT rules starting from before the if statement S_{if} to the start of the then branch S_{thn} or the start of the else branch S_{els} . So the **nia** in the hypotheses of the rule map to S_{if} and the one in the conclusion to S_{thn} (and S_{els}). We then connect the state at the end of the two branches E_{thn} and E_{els} with the state at the end of the if statement with the unconstrained implications $E_{thn,els} \implies E_{if}$ as we did for the reachability analysis.

4.1.2 Implementation

The **nia** subfolder of the course repository contains a template for the implementation of the Horn-clause-based abstraction of **nia**.

- <https://gitlab.secpriv.tuwien.ac.at/teaching/fmsp-s26/-/tree/main/nia>

Specifically, the **nia.py** file contains the skeleton of the implementation of the abstraction function, which, similarly to the **alpha.py** file, returns a set of horn-clauses that model the analysis. The abstraction function implements the procedure described in Sec. 4.1.1. We marked all the non-implemented functions and sections with the **PROJECT TODO** comment.

Your task is to implement the missing parts of the **nia.py** file, following the definition of the **nia** rules of Fig. 1. The abstraction function can be tested against a set of positive and negative examples by running the **nia.py** script. For more information about the structure of the template, refer to the **README.md** file.

Submission (10pt). Add to your submission (only) the **nia.py** file containing the abstraction function implementing the **nia** rules. The abstraction function should pass all tests in the **tests/nia** subfolders.

5 Relaxing Noninterference

5.1 Example Program: Sealed-Bid Auction

Let us now focus on the example program shown in Figure 4. The program receives two private bids and publishes only the outcome of the auction before resetting the bid variables to 0. Following the convention of **nia.py**, we annotate the initial and expected security level of the variables by prefixing the variable name with **h** or **l**. Specifically, the bids are secret inputs, whereas the auction result is a public output.

The program compares the two bids and stores in **lwinner** whether bidder A wins, bidder B wins, or the bids tie. From this description it is clear that the exact values of the bids remain secret, while the winner intentionally reveals a relation between them. In the program below, **lwinner = 0** denotes *A wins*, **lwinner = 1** denotes *B wins*, and **lwinner = 2** denotes a tie.

```

1  lwinner = -1;
2
3  IF hbidA < hbidB THEN
4      lwinner = 1
5  ELSE
6      IF hbidB < hbidA THEN
7          lwinner = 0
8      ELSE
9          lwinner = 2
10     END
11 END;
12
13 hbidA = 0;
14 hbidB = 0

```

Figure 4: A program that publicly reveals only the outcome of a sealed-bid auction.

Submission (2pt). Discuss (in sec. 5.1 of `submission.pdf`) the output of the analysis of the program in Fig. 4, which rules of `nia` are involved to detect interference, and why.

5.2 Controlled Information Leaks: Abstract Noninterference

In the program of Fig. 4, it may be considered acceptable to disclose who won the auction, provided that the exact bid amounts remain confidential. However, traditional noninterference is too restrictive in this context, as it would classify the program as insecure due to the dependency of the public outputs on the secret bids.

Inspired by the work of Giacobazzi et al. [1], we can relax our definition of noninterference by defining an abstract observation that models the attacker’s ability to distinguish different executions of the program.

In particular, instead of requiring that all secret values remain indistinguishable, we require indistinguishability up to a pair of input and output abstraction functions. The input abstraction ϕ allows us to partition the set of memories into equivalence classes that represent the amount of information we allow to be leaked. In this example, the abstraction depends on the relation between two secret variables rather than on one variable in isolation. We could apply the same reasoning for the public data, using an output abstraction function ρ .

We can thus define abstract noninterference as follows.

$$\begin{aligned}
 &\forall m_1, m_2. \\
 &\quad \phi(m_1) = \phi(m_2) \wedge \\
 &\quad \langle c, m_1 \rangle \xrightarrow{*} \langle \epsilon, m'_1 \rangle \wedge \langle c, m_2 \rangle \xrightarrow{*} \langle \epsilon, m'_2 \rangle \implies \\
 &\quad \rho(m'_1) = \rho(m'_2)
 \end{aligned}$$

This statement corresponds to the definition of noninterference of Sec. 4.1 if we consider

$$\phi(m) = \rho(m) = \lambda v. \begin{cases} \top & \text{if } \Gamma(v) = H \\ m(v) & \text{if } \Gamma(v) = L \end{cases}$$

that is, if both abstraction functions map to the low part of the memory, turning the equality of the abstractions into low equality.

With this new formulation of noninterference, we can define ϕ so that it preserves the low part of the memory and abstracts the two secret bids to their comparison outcome.

$$\phi(m) = \lambda v. \begin{cases} \text{winner}(m) & \text{if } v = \text{hbidA} \vee v = \text{hbidB} \\ m(v) & \text{if } \Gamma(v) = L \\ \top & \text{otherwise} \end{cases}$$

where

$$\text{winner}(m) = \begin{cases} 0 & \text{if } m(\text{hbidB}) < m(\text{hbidA}) \\ 1 & \text{if } m(\text{hbidA}) < m(\text{hbidB}) \\ 2 & \text{otherwise} \end{cases}$$

returns the representative of the comparison outcome of the two bids. Since we do not need an output abstraction, we can keep ρ equal to the low-part projection of traditional noninterference.

This way, the program of Fig. 4 is (abstractly) noninterferent, since we do not leak the precise values of the bids but only their comparison outcome. Note that the abstraction above **must** align exactly with the distinctions made by the program, including the treatment of ties; otherwise, deviations may lead to observable differences and thus interference.

5.2.1 Implementation

Implement the above extension to noninterference in the abstraction function of the `nia-abstract.py` script by parameterizing the relaxed analysis with program-specific memory abstraction.

In the Horn-clause encoding, the taint propagation rules remain unchanged: the only difference is how the analysis decides which executions may be related. Instead of hard-coding a special case for a particular variable, the relaxed analysis should require

$$\phi(m_1) = \phi(m_2)$$

whenever two runs are treated as members of the same admissible input class (JOIN rule). The default strict analysis is recovered by instantiating ϕ with the low projection, whereas the auction abstraction above should make the program in Fig. 4 provably secure.

In the `nia.py` file we included commented snippets for testing the abstraction against a few variations of the above program. Previous test cases for strict noninterference should still pass.

Submission (6pt). Implement a relaxed noninterference analysis by modifying your `nia.py`. Add to your submission the modified script as `nia-abstract.py`. The new script should pass all tests under the `tests/abstract` folder, support both the default low equality and the auction abstraction above, and return `sat` for the program in Fig. 4. Discuss your implementation in sec. 5.2 of `submission.pdf`; explain in particular, why `tests/abstract/pos/simple_winner.while` is labeled (wrongly) as interferent and how the analysis could be modified to label it correctly.

References

- [1] R. Giacobazzi and I. Mastroeni, “Abstract non-interference: A unifying framework for weakening information-flow,” *ACM Trans. Priv. Secur.*, vol. 21, no. 2, Feb. 2018, doi: 10.1145/3175660.